

TP2

UPB

2025-2026

Table des matières

1	Consignes générales	2
2	TP 2-1 : Topic Modeling et Prétraitement	2
2.1	Objectifs pédagogiques	2
2.2	Données	2
2.3	Énoncé	2
2.4	Livrables attendus	3
3	TP 2-2 : Word Embeddings et Représentations Denses	4
3.1	Objectifs pédagogiques	4
3.2	Données	4
3.3	Énoncé	4
3.4	Livrables attendus	5
4	TP 2-3 : Génération de Texte et Reconnaissance d'Entités Nommées	6
4.1	Objectifs pédagogiques	6
4.2	Données	6
4.3	Énoncé	6
4.4	Livrables attendus	7
5	TP 2-4 : Modèles Sequence-to-Sequence et Question-Réponse	8
5.1	Objectifs pédagogiques	8
5.2	Données	8
5.3	Énoncé	8
5.4	Livrables attendus	9
6	TP 2-5 : Transformers et Large Language Models	10
6.1	Objectifs pédagogiques	10
6.2	Données	10
6.3	Énoncé	10
6.4	Livrables attendus	11

1 Consignes générales

- TP par groupe
- Python obligatoire
- Justifications mathématiques et interprétations NLP requises selon les livrables attendus
- Le rendu de vos travaux sera sous la forme :
 - soit un Jupyter Notebook en python, qu'il sera possible de rouler cellule par cellule
 - soit un fichier .py qu'il faudra rouler de la manière suivante **python3 fichier.py**
 - vos scripts devront être commenté
- Chaque groupe recevra un mail auquel il faudra envoyer les travaux des TP que vous aurez exécuter, ce sera donc un seul envoi par groupe.

2 TP 2-1 : Topic Modeling et Prétraitement

2.1 Objectifs pédagogiques

- Maîtriser le prétraitement d'un corpus textuel (nettoyage, normalisation, tokenisation).
- Implémenter et interpréter un modèle LDA (Latent Dirichlet Allocation).
- Comparer LDA avec la factorisation par matrices non-négatives (NMF).
- Évaluer l'impact du prétraitement sur l'interprétabilité des topics.

2.2 Données

Dataset IMDB : 50 000 critiques de films (fichier `IMDBDataset.csv`).

2.3 Énoncé

Partie A : Prétraitement

1. Charger le dataset IMDB avec `pandas`. Afficher les 10 premières lignes et la distribution des labels (`positive/negative`).
2. Créer une fonction `preprocess_text(text)` qui réalise les opérations suivantes :
 - Conversion en minuscules ;
 - Suppression des balises HTML (`<br />`, etc.) ;
 - Suppression des caractères spéciaux et ponctuation ;
 - Tokenisation simple (`split()` ou NLTK) ;
 - Suppression des stopwords (liste NLTK anglaise) ;
 - Lemmatisation (`WordNetLemmatizer` de NLTK).
3. Appliquer cette fonction à la colonne `review` et stocker le résultat dans `review_clean`. Afficher un exemple avant/après.
4. Vectoriser le corpus prétraité avec `CountVectorizer` (`max_features=5000`, `max_df=0.95`, `min_df=2`). Afficher la forme de la matrice document-terme.

Partie B : LDA avec `scikit-learn`

1. Instancier un modèle `LatentDirichletAllocation` avec `n_topics=10`, `learning_method='online'`, `random_state=42`, `max_iter=10`.
2. Entraîner le modèle sur la matrice document-terme.
3. Écrire une fonction `display_topics(model, feature_names, no_top_words)` qui affiche, pour chaque topic, les 10 mots les plus probables.

4. Interpréter les 10 topics obtenus : associez-vous un thème à chacun ?
5. Calculer la perplexité du modèle sur le corpus d'entraînement.

Partie C : NMF et comparaison

1. Vectoriser le corpus prétraité avec `TfidfVectorizer` (mêmes contraintes que ci-dessus).
2. Instancier un modèle NMF avec `n_components=10`, `random_state=42`, `max_iter=500`.
3. Entraîner et afficher les topics NMF avec la même fonction.
4. Comparer qualitativement les topics LDA et NMF. Lequel semble plus interprétable ? Pourquoi ?
5. **Question théorique** : Expliquez la différence fondamentale entre LDA (approche bayésienne génératrice) et NMF (factorisation algébrique). Dans quel cas privilégier l'une ou l'autre ?

Partie D : Exploration avec Gensim

1. Convertir le corpus prétraité en format Gensim (liste de listes de tokens).
2. Créer un dictionnaire et filtrer les extrêmes (`filter_extremes(no_below=100, no_above=0.5)`).
3. Entraîner un modèle `LdaModel` Gensim avec 10 topics.
4. Comparer les résultats avec ceux de scikit-learn. Que constatez-vous sur l'impact du pré-traitement ?

2.4 Livrables attendus

- Notebook Python exécutable contenant le code, les visualisations (WordCloud optionnel) et les commentaires interprétatifs.
- Une cellule markdown résumant la comparaison LDA vs NMF en 5 points.

3 TP 2-2 : Word Embeddings et Représentations Denses

3.1 Objectifs pédagogiques

- Comprendre et implémenter Word2Vec (Skip-Gram et CBoW) avec Gensim.
- Visualiser les embeddings en 2D (PCA, t-SNE).
- Manipuler les embeddings pré-entraînés GloVe.
- Découvrir Doc2Vec pour la représentation de documents entiers.

3.2 Données

Corpus d'économie (extrait du manuel du cours) ou dataset IMDB prétraité du TP1.

3.3 Énoncé

Partie A : Word2Vec from scratch avec Gensim

1. À partir du corpus IMDB prétraité (TP1), préparer une liste de phrases tokenisées pour Gensim.
2. Entraîner un modèle Word2Vec avec les paramètres suivants :
 - `vector_size=100, window=5, min_count=10, sg=1` (Skip-Gram), `negative=5, epochs=5`.
3. Afficher le vecteur du mot "movie". Quelle est sa dimension ?
4. Trouver les 5 mots les plus similaires à "movie", "good", "bad" en utilisant la similarité cosinus.
5. Tester l'analogie : `king - man + woman = ?` (si le vocabulaire le permet, sinon `good - bad + terrible = ?`).
6. Ré-entraîner un modèle avec `sg=0` (CBoW). Comparer la similarité des mots précédents. CBoW ou Skip-Gram semble-t-il meilleur sur ce corpus ?

Partie B : Visualisation des embeddings

1. Extraire les vecteurs des 200 mots les plus fréquents du vocabulaire.
2. Réduire la dimension à 2 avec PCA, puis avec t-SNE (`perplexity=30, n_iter=1000`).
3. Tracer un nuage de points où chaque point est un mot, coloré par cluster (utiliser `KMeans` avec `n_clusters=5` sur les vecteurs complets).
4. Annoter quelques mots représentatifs sur le graphique. Observez-vous des regroupements sémantiques ?

Partie C : GloVe : embeddings pré-entraînés

1. Télécharger les embeddings GloVe 6B (dimension 50 ou 100) depuis la source officielle Stanford.
2. Écrire une fonction `load_glove_embeddings(filepath)` qui charge les vecteurs dans un dictionnaire Python.
3. Afficher le vecteur de "computer" et ses 5 plus proches voisins (distance euclidienne ou cosinus).
4. Comparer le voisinage de "computer" entre Word2Vec (entraîné sur IMDB) et GloVe. Quelles différences observez-vous ? Expliquez.

Partie D : Doc2Vec

1. Préparer les données au format `TaggedDocument` de Gensim (chaque critique reçoit un tag unique).
2. Entraîner un modèle Doc2Vec (`vector_size=100, window=5, min_count=5, epochs=20`).

3. Calculer la similarité cosinus entre le document tagué 0 et le document tagué 1. Interpréter.
4. Inférer un vecteur pour un nouveau document (une critique fictive que vous rédigerez) avec `model.infer_vector()`. Trouver les 3 documents du corpus les plus similaires à ce nouveau texte.

3.4 Livrables attendus

- Notebook avec les visualisations PCA/t-SNE.
- Tableau comparatif Word2Vec Skip-Gram / CBoW / GloVe / Doc2Vec (forces, faiblesses, cas d'usage).

4 TP 2-3 : Génération de Texte et Reconnaissance d'Entités Nommées

4.1 Objectifs pédagogiques

- Construire un générateur de texte au niveau caractère avec LSTM (Keras).
- Comprendre les limites des RNN unidirectionnels et l'intérêt des BRNN.
- Implémenter un tagger NER bidirectionnel (BiLSTM + TimeDistributed).

4.2 Données

- *Harry Potter and the Sorcerer's Stone* (texte brut pour la génération).
- Corpus NER de Kaggle (format CoNLL-like : token, POS, NER tag).

4.3 Énoncé

Partie A : Génération de texte par LSTM

1. Charger le texte intégral de *Harry Potter* (téléchargement ou fichier local). Supprimer les caractères non-ASCII.
2. Construire un dictionnaire de caractères uniques. Afficher la taille du vocabulaire V .
3. Créer des séquences glissantes de longueur $L = 100$: $X = [c_t, \dots, c_{t+L-1}]$, $y = c_{t+L}$ encodé en one-hot.
4. Normaliser X en divisant par V .
5. Construire un modèle séquentiel Keras :
 - Couche LSTM (128 unités, `return_sequences=False`);
 - Couche Dense(V , activation `softmax`).
6. Compiler avec `categorical_crossentropy` et Adam. Entraîner sur 30 époques (`batch_size=128`).
7. Écrire une fonction `generate_text(seed_text, length=200)` qui :
 - Encode le `seed_text`;
 - Prédit le caractère suivant;
 - Fait du *sampling* (softmax avec température $T = 0.5$);
 - Concatène et itère.
8. Générer 3 textes à partir de graines différentes (ex. "Harry", "The magic", "Professor"). Commenter la cohérence orthographique et syntaxique.
9. **Question** : Pourquoi le texte généré contient-il des fautes et des mots inventés ? Proposez 3 améliorations architecturales ou data-centric.

Partie B : NER avec BiLSTM

1. Charger le corpus NER. Parser le fichier pour obtenir une liste de phrases, chaque phrase étant une liste de tuples (`token`, `pos`, `ner_tag`).
2. Construire les vocabulaires : `word2idx`, `tag2idx`. Ajouter un token PAD (index 0).
3. Pad les séquences de tokens et de tags à la longueur maximale de la phrase (`max_len`) avec `keras.preprocessing.sequence.pad_sequences`.
4. One-hot encoder les tags NER (`to_categorical`).
5. Construire le modèle :
 - Couche `Embedding(input_dim=n_words, output_dim=50, input_length=max_len)`;
 - Couche `Bidirectional(LSTM(100, return_sequences=True))`;

- Couche `TimeDistributed(Dense(n_tags, activation='softmax'))`.
- 6. Compiler et entraîner (20 époques, `validation_split=0.1`).
- 7. Afficher la matrice de confusion (token-level) sur l'ensemble de test.
- 8. **Question :** Pourquoi utilise-t-on `TimeDistributed(Dense)` plutôt qu'une simple couche `Dense` ? Expliquez le rôle du `return_sequences=True` dans la BiLSTM.

4.4 Livrables attendus

- Notebook avec les textes générés et leur analyse critique.
- Courbes de loss du modèle NER et matrice de confusion.

5 TP 2-4 : Modèles Sequence-to-Sequence et Question-Réponse

5.1 Objectifs pédagogiques

- Comprendre l'architecture Encodeur-Décodeur (Seq2Seq).
- Implémenter un modèle Seq2Seq avec l'API fonctionnelle Keras.
- Appliquer le Seq2Seq à une tâche de Question-Réponse (caractère par caractère).
- Maîtriser le *teacher forcing* et le décalage de la cible.

5.2 Données

Dataset SQuAD (Stanford Question Answering) simplifié : paires (question, réponse) extraites.

5.3 Énoncé

Partie A : Prétraitement SQuAD

1. Charger le fichier JSON SQuAD. Extraire les questions et les réponses (texte, pas les indices).
2. Filtrer les paires où la réponse fait moins de 50 caractères et la question moins de 100.
3. Construire deux vocabulaires de caractères : `vocab_enc` (questions) et `vocab_dec` (réponses). Ajouter les tokens spéciaux `<START>` et `<END>`.
4. Encoder les questions et réponses en séquences d'entiers. Pad à des longueurs fixes.
5. Créer la cible décalée (*teacher forcing*) : la séquence d'entrée du décodeur est la réponse préfixée par `<START>`, la cible est la réponse suffixée par `<END>`.
6. One-hot encoder les entrées et les cibles en 3D.

Partie B : Architecture Seq2Seq

1. Définir l'**encodeur** avec l'API fonctionnelle Keras :
 - Entrée : `Input(shape=(max_len_question, vocab_enc_size))`;
 - `LSTM(256, return_state=True)` – récupérer `output`, `state_h`, `state_c`.
2. Définir le **décodeur** :
 - Entrée : `Input(shape=(max_len_answer, vocab_dec_size))`;
 - `LSTM(256, return_sequences=True, return_state=True, initial_state=[state_h, state_c])`.
3. Ajouter une couche `Dense(vocab_dec_size, activation='softmax')` sur chaque pas de temps (`TimeDistributed` implicite via `return_sequences=True`).
4. Définir le modèle complet `model = Model([encoder_inputs, decoder_inputs], decoder_outputs)`.
5. Compiler et entraîner (`batch_size=64`, 20 époques).

Partie C : Inférence et évaluation

1. Construire le modèle d'inférence (encodeur et décodeur séparés).
2. Écrire une fonction `answer_question(input_seq)` qui :
 - Encode la question pour obtenir les états `c`;
 - Initialise le décodeur avec `<START>`;
 - Génère la réponse caractère par caractère jusqu'à `<END>` ou longueur max.
3. Tester sur 5 questions du dataset. Afficher la question, la réponse attendue et la prédiction.
4. Calculer la métrique `accuracy exact-match` (réponse prédite == réponse exacte) sur un échantillon de 100 paires.

Partie D : Analyse critique

1. **Question :** Expliquez le rôle du vecteur de contexte c transmis de l'encodeur au décodeur. Que se passe-t-il si la phrase source est très longue (problème du *bottleneck*) ?
2. Proposez une solution à ce problème (indice : mécanisme d'attention).

5.4 Livrables attendus

- Notebook avec le code complet de l'inférence.
- Tableau des 5 prédictions (question / attendu / prédit).

6 TP 2-5 : Transformers et Large Language Models

6.1 Objectifs pédagogiques

- Comprendre l'architecture Transformer (self-attention, positional encoding).
- Utiliser la bibliothèque `transformers` (HuggingFace) pour le fine-tuning.
- Implémenter un classifieur de sentiment avec BERT.
- Explorer les mécanismes d'attention d'un modèle pré-entraîné.

6.2 Données

Dataset IMDB (sous-échantillon de 5 000 critiques pour des raisons de temps de calcul).

6.3 Énoncé

Partie A : Prétraitement pour BERT

1. Installer `transformers` et `torch` (ou `tensorflow`).
2. Charger le tokenizer `BertTokenizer.from_pretrained('bert-base-uncased')`.
3. Tokenizer une critique d'exemple avec `tokenizer(text, padding=True, truncation=True, max_length=128, return_tensors='pt')`. Afficher les `input_ids`, `attention_mask` et `token_type_ids`.
4. Expliquer à quoi sert chacun de ces trois tenseurs.

Partie B : Fine-tuning BERT pour la classification

1. Charger `BertForSequenceClassification.from_pretrained('bert-base-uncased', num_labels=2)`.
2. Préparer un sous-échantillon IMDB (2 500 train / 2 500 test). Encoder les textes et labels en `torch_datasets` ou `tf_datasets`.
3. Configurer l'entraînement :
 - Optimiseur : `AdamW (learning_rate=2e-5)`;
 - Nombre d'époques : 3;
 - Batch size : 16 (ou 8 selon la RAM).
4. Entraîner le modèle. Afficher la loss et l'accuracy à chaque époque.
5. Évaluer sur l'ensemble de test. Afficher la matrice de confusion et le rapport de classification (précision, rappel, F1).

Partie C : Exploration de l'attention

1. Choisir une phrase test (ex. "The movie was great but the ending was disappointing").
2. Passer cette phrase dans le modèle BERT et extraire les poids d'attention de la dernière couche (`outputs.attentions`).
3. Visualiser la matrice d'attention (heatmap) pour la tête 0 de la dernière couche. Quels mots attirent le plus l'attention du mot "ending" ?
4. Comparer avec la tête 5 (ou une autre). Observez-vous des rôles différents ?

Partie D : Comparaison des architectures Transformer

1. Créer un tableau comparatif (Markdown ou LaTeX dans le notebook) des trois familles :
 - Encoder-only (BERT) : attention bidirectionnelle, MLM, classification/NER.
 - Decoder-only (GPT) : attention masquée, génération auto-régressive.
 - Encoder-Decoder (T5) : attention croisée, traduction/résumé.
2. **Question** : Pourquoi le positional encoding est-il indispensable dans le Transformer alors que les LSTM n'en ont pas besoin ?
3. **Question** : Expliquez le rôle du facteur $\sqrt{d_k}$ dans le scaled dot-product attention. Que se passerait-il sans cette mise à l'échelle ?

6.4 Livrables attendus

- Notebook avec le modèle fine-tuné et les courbes d'entraînement.
- Heatmap de l'attention et analyse écrite.
- Tableau comparatif des architectures.